Structured MUS Explanations: Comparing Graph-Based and Trace-Based Representations

Luis Quesada[1][0000-0003-3177-655X], Helmut Simonis[1][0009-0003-6117-2463], and Barry O'Sullivan[1][0000-0002-0090-2085]

Insight Research Ireland Centre for Data Analytics
School of Computer Science & IT, University College Cork, Ireland
{luis.quesada,helmut.simonis,barry.osullivan}@insight-centre.org

Abstract. Minimal Unsatisfiable Subsets (MUSes) provide logically minimal justifications for derived literals in constraint programming, but remain flat objects that do not expose the internal structure of the reasoning. We introduce structured explanation graphs that represent dependency relations between MUS-derived deductions. The framework combines a lemma-based view, which organises deductions through reusable intermediate literals, with a complementary rule-based view that highlights the structural constraints responsible for contradictions. We further relate these graph-based explanations to operational explanation traces by interpreting the graphs as partial-order representations whose topological linearisations induce explanation traces. Experiments on Sudoku benchmarks and reconstructed Demystify traces show strong agreement between the dependency orders induced by the explanation graphs and the corresponding operational traces. The results additionally show that graph-based explanations provide a substantially more compact representation of reasoning structure than fully linearised operational traces. The results also show that bottom-up construction tends to produce more coherent explanations and, with the native CP portfolio used in this evaluation, is faster than condensed construction on most of the benchmark instances.

Keywords: Constraint Programming · Explanations · MUS · Sudoku · Explainability

1 Introduction

Explanations are active to modern Constraint Programming (CP) [19], supporting conflict analysis, learning, debugging, and interactive solving [9]. A common logical foundation is the notion of a *Minimal Unsatisfiable Subset* (MUS): a minimal set of constraints whose joint presence implies a contradiction [15,11]. MUSes provide logically minimal explanations that are independent of search order [15,14]. However, they remain fundamentally flat objects: they identify

which constraints participate in a contradiction, but not how the corresponding deductions depend on one another.

In structured domains such as Sudoku, deductions arise through interactions between row, column, and block constraints. Flat MUSes capture only minimal support sets, whereas operational systems such as Demystify produce linear deduction traces through sequences of human-interpretable reasoning steps [1,6]. A graph-based representation is therefore not merely another trace: it compactly represents the family of traces obtained by topologically linearising the dependency partial order. From such a graph one may recover many admissible total orders, and may choose linearisations that respect additional user-specific preferences, such as favouring local moves, particular rule types, or shorter supports. Conversely, a single linear trace records only one presentation order and does not, by itself, separate essential dependencies from arbitrary ordering choices. This motivates graph-based representations that retain the fundamental reasoning dependencies explicitly as partial orders between deductions.

In this paper, we lift flat MUSes into structured explanation graphs for a Sudoku CSP model. For each derived literal, we compute a MUS and organise the resulting information under two complementary views. In the *lemma-based view*, nodes represent literals and edges encode justification dependencies. Each non-leaf literal is justified by a MUS, and derived literals may be reused across explanations. In the *rule-based view*, explanations are organised around rule applications. Each MUS is represented as a three-layer graph: Literals $\rightarrow$ Rules $\rightarrow$ $\bot$, where literals connect to the Sudoku constraints they participate in (e.g., `AllDiffRow`, `AllDiffBlock`), and rules connect to a contradiction node. There are no literal-to-literal or rule-to-rule edges. Reuse therefore occurs at the level of structural constraints. The two views reorganise the same MUS information differently: the lemma-based view highlights reuse of derived literals, while the rule-based view highlights reuse of structural constraints. This dual perspective underlies our admissibility policy.

Figure 1 illustrates the type of reasoning step we aim to capture structurally. The highlighted hints, together with three structural constraints, suffice to derive a contradiction under the counterfactual assumption $\neg(X_{1,E} = 6)$. Viewed as a flat MUS, this explanation is simply a minimal set of constraints. Viewed structurally, however, it exposes how row, column, and block constraints interact through a small collection of literals. Our goal is to make this dependency structure explicit and reusable. [1]

We study two construction strategies. The *condensed construction* introduces intermediate lemmas during recursive decomposition. The *bottom-up construction* iteratively admits literals whose smallest MUS explanations satisfy structural criteria derived from the rule-based view. A literal is admitted only if its explanation (i) involves fewer than nine literals and (ii) forms a connected three-layer rule-based graph. These criteria aim to capture coherent reasoning units rather than arbitrary minimal cores. While condensation reduces duplication,

[1] The appendix provides additional examples together with detailed explanations of both positive and negative targets.

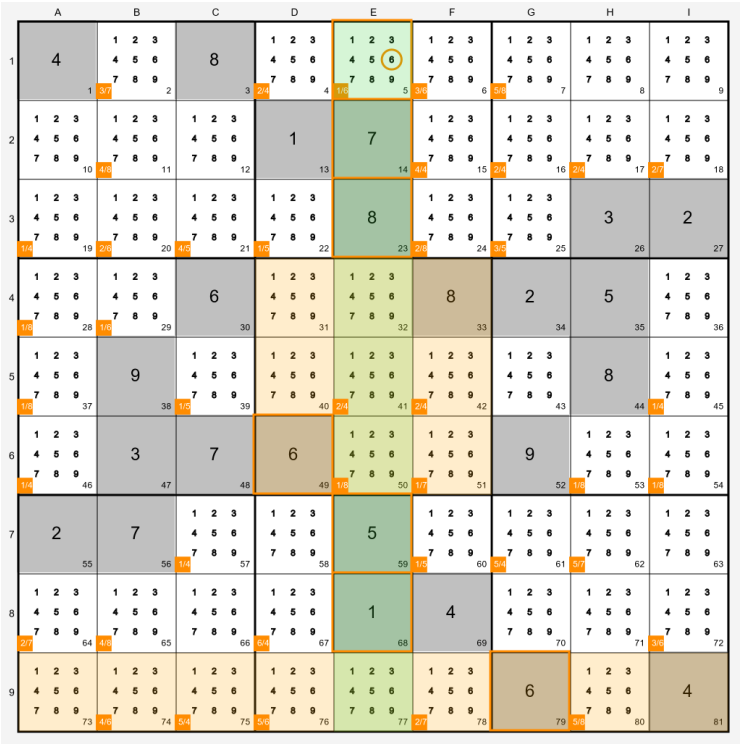

Fig. 1. Motivating example. The shaded cells correspond to `AllDiffCol`(E), `AllDiffBlock`(5), and `AllDiffRow`(9). Together with the highlighted hints $X_{2,E} = 7$, $X_{3,E} = 8$, $X_{6,D} = 6$, $X_{7,E} = 5$, $X_{8,E} = 1$, and $X_{9,G} = 6$, these rules explain why $X_{1,E} \neq 6$.

our experiments show that bottom-up construction guided by connectivity filtering yields explanations that align more closely with recognisable reasoning steps.

The resulting explanation graphs represent partial dependency orders between deductions. Concrete explanation traces are then obtained by selecting topological linearisations of these graphs. This perspective makes it possible to compare graph-based representations with operational MUS-based traces produced by systems such as Demystify.

The contributions of this paper are:

- a formal MUS-based explanation framework for Sudoku CSP derivations,
- lemma-based and rule-based explanation graph constructions,
- a rule-based connectivity criterion for admissibility,
- a bottom-up explanation strategy guided by this criterion,
- a trace extraction framework relating graph-based and trace-based explanations,

– and an empirical comparison between condensed and bottom-up constructions, including a comparison with reconstructed Demystify-style operational traces.

Our results demonstrate that minimality alone is insufficient for useful explanations: structural criteria are essential for producing coherent explanation graphs and meaningful operational traces.

2 Related Work

MUSes, diagnosis, and SMUS extraction. MUSes (minimal unsatisfiable subsets) are a standard explanation object in logic and model-based diagnosis, where inconsistencies are characterised by minimal conflicting constraint sets. This perspective originates in Reiter's framework [18] and underlies much of modern SAT and CP practice. On the algorithmic side, practical MUS extraction and optimisation have been studied extensively [15], including smallest MUS (SMUS) computation via hitting-set dualisation [11]. QuickXplain-style procedures provide a general mechanism for isolating subset-minimal conflicts in over-constrained systems [12]. Our work builds on this MUS/SMUS tradition, but focuses on organising MUSes into reusable structured explanations rather than treating them as isolated flat cores.

Explanations in constraint programming. Explanations have long been used in CP to justify propagation, support debugging, and enable learning. Explanation-based constraint programming was introduced in early work by Cambazard and Jussien [2]. A major line of research is *lazy clause generation* (LCG), where propagation steps are explained as clauses and reused for learning [17,5]. These approaches primarily target solver performance and clause learning, whereas our work focuses on concise MUS-based explanations with explicit dependency structure. Gocht et al. proposed an auditable CP solver producing independently verifiable proof logs of the solving process [7]. Such proof traces provide global correctness certificates, but are not designed to expose reusable MUS-based reasoning structure or compact human-oriented explanations. Recent work has also emphasised the importance of understanding how humans reason about constraint problems and how CP tools can provide more intelligible explanations to their users [10].

Operational and step-wise explanations. Several recent approaches aim to transform flat unsatisfiable subsets into sequences of local reasoning steps. Bogaerts et al. propose a general framework for step-wise CSP explanations [1], and Gamba et al. study explanation computation via unsatisfiable subset optimisation [6]. These approaches focus on constructing operational explanation traces in which deductions are presented sequentially. In the domain of pen-and-paper puzzles, Espasa et al. introduce the Demystify framework, which uses MUS-based reasoning to generate human-oriented step-by-step explanations for Sudoku, Futoshiki, Star Battle, and related puzzles [4]. Demystify produces operational traces consisting of local MUS-based deduction steps together with explicit candidate eliminations. Our work instead focuses on graph-based representations

that expose dependency structure and reusable intermediate deductions. Operational traces are then obtained as linearisations of these dependency graphs. Korekawa et al.'s notion of solution paths for pencil puzzles similarly models solving as a progression of deterministic deductions rather than branching search [13]. Our work differs in that the intermediate deductions are represented through explicit MUS-based dependency structures. Operational traces are then obtained as linearisations of these dependency graphs.

Sudoku as a benchmark domain. Sudoku has frequently served as a benchmark domain for explainable constraint solving. It has been studied both as a modelling benchmark [20] and as an application domain for explanation and interactive support [8]. Eppstein's work on single-digit Sudoku subproblems further illustrates Sudoku as a domain for studying human-style deductive reasoning, including Nishio-style global consistency reasoning [3]. We adopt Sudoku as a structured CSP setting in which reasoning steps are easily interpretable and rule granularity is well understood.

SAT encodings and rule granularity. Standard CNF encodings of Sudoku are well established [16]. We use selector literals to group clauses corresponding to the same structural rule, enabling rule-level MUS extraction. Our contribution is therefore not the encoding itself, but the explicit rule grouping that bridges CNF-level cores and human-facing structural rule violations.

3 Sudoku CSP Model and SAT Encoding

We model standard 9×9 Sudoku as a constraint satisfaction problem (CSP) $\langle X, D, C \rangle$.

3.1 Variables and Domains

For each cell (r, c) with $r, c \in \{1, \ldots, 9\}$, we introduce a variable $X_{r,c}$ with domain

$$D(X_{r,c}) = \{1, \ldots, 9\}.$$

A puzzle instance provides a set of given clues. Each clue fixes the value of a variable and is represented as a unary constraint of the form $X_{r,c} = v$. We denote by H the set of such hints.[2]

Reasoning steps manipulate literals of the form $X_{r,c} = v$ (value assignments) and $X_{r,c} \neq v$ (candidate eliminations). These literals form the basic explanation objects used throughout the paper.

3.2 Constraints

The constraint set C consists of the standard Sudoku `AllDifferent` constraints:

[2] In the automatically generated figures we use the notation $x[r, c] = v$ to denote the assignment $X_{r,c} = v$ used in the text.

- 9 row constraints,
- 9 column constraints,
- 9 block constraints.

Each constraint enforces that the variables in the corresponding unit take distinct values. Together these 27 constraints capture the structural rules of the puzzle.

3.3 SAT Encoding

For MUS extraction we translate the Sudoku model into propositional form using the standard SAT encoding for Sudoku [16]. For each triple (r, c, v) we introduce a Boolean variable $x_{r,c,v}$ with intended meaning

$$x_{r,c,v} \Leftrightarrow (X_{r,c} = v).$$

The encoding contains clauses enforcing the standard Sudoku constraints.

Cell constraints (exactly one value). For every cell (r, c) we enforce that exactly one value is assigned:

$$\bigvee_{v=1}^{9} x_{r,c,v} \qquad \text{and} \qquad \bigwedge_{1 \le v < w \le 9} (\neg x_{r,c,v} \vee \neg x_{r,c,w}).$$

Row constraints. For every row r and value v, value v appears in at most one column:

$$\bigwedge_{1 \le c < d \le 9} (\neg x_{r,c,v} \vee \neg x_{r,d,v}).$$

Column constraints. For every column c and value v, value v appears in at most one row:

$$\bigwedge_{1 \le r < s \le 9} (\neg x_{r,c,v} \vee \neg x_{s,c,v}).$$

Block constraints. Let B range over the nine 3×3 blocks and let $(r, c) \in B$ denote the cells of block B. For every block B and value v, value v appears in at most one cell of B:

$$\bigwedge_{\substack{(r,c),(s,d) \in B \\ (r,c) < (s,d)}} (\neg x_{r,c,v} \vee \neg x_{s,d,v}).$$

Hints. Each clue $X_{r,c} = v$ is encoded as the unit clause $x_{r,c,v}$.

The resulting CNF formula forms the basis for the MUS and SMUS computations used throughout the explanation framework.

3.4 Structural Rules

For explanation purposes, each structural Sudoku constraint is treated as a *rule*. In particular we consider the following rule families:

- `AllDiffRow`(r) for each row r,
- `AllDiffCol`(c) for each column c,
- `AllDiffBlock`(b) for each 3×3 block b.

This yields 27 structural rules, each corresponding to one `AllDifferent` constraint of the CSP model. When translated to propositional form, a rule expands into a set of CNF clauses.

To support rule-level explanations, clauses associated with the same structural rule are grouped using selector literals. This allows MUS and SMUS computations to refer directly to row, column, and block rules rather than only to individual CNF clauses. In the rule-based explanation view (Section 6), explanations therefore identify which structural rules participate in a contradiction.

4 MUS-Based Explanations

We formalise explanations as minimal inconsistent subsets of assumptions relative to the Sudoku constraints.

Explanation problem. Let C denote the Sudoku constraints and let A denote a set of assumptions. Depending on the context, these assumptions may correspond to original puzzle hints, previously established literals, rule selectors, or accumulated trace-state facts. For a literal ℓ (either $X_{r,c} = v$ or $X_{r,c} \neq v$), we consider the counterfactual formula

$$C \cup A \cup \{\neg\ell\}.$$

If this formula is unsatisfiable, then ℓ is logically implied by $C \cup A$. Intuitively, the explanation problem asks which subset of the available assumptions suffices to make the negation of ℓ inconsistent with the Sudoku rules.

A subset $A' \subseteq A$ is an *explanation* for ℓ if

$$C \cup A' \cup \{\neg\ell\}$$

is unsatisfiable.

MUS and SMUS. An explanation A' is a *minimal unsatisfiable subset* (MUS) if it is unsatisfiable and no strict subset of A' is. Among all MUSes, a *smallest MUS* (SMUS) is one with minimum cardinality. Multiple MUSes may exist for the same literal.

Extraction backends. We compute explanations using two standard procedures:

- **QuickXplain** [12] returns a subset-minimal MUS by treating the Sudoku rules together with $\neg\ell$ as hard constraints and the assumptions as soft constraints.

– **OptUx** [11] computes a smallest MUS (SMUS) via SAT-based hitting-set dualisation. In this encoding, Boolean variables $x_{r,c,v}$ represent value assignments to cells, the Sudoku rules are translated into CNF clauses, and assumptions are encoded as soft clauses. The explanation problem then corresponds to computing a smallest unsatisfiable subset of these soft clauses.

Both backends produce flat MUS explanations, which we subsequently lift into structured explanation graphs. Their runtime and size characteristics are compared in Section 10.

5 Lemma-Based Explanation Graphs

We lift a flat MUS explanation into a structured directed graph. Let t be an assignment literal justified by a MUS $G \subseteq H$ with respect to $C \cup \{\neg t\}$. We call G the *base MUS* of t. The base MUS remains fixed throughout construction.

Graph structure. A lemma-based explanation graph is a directed acyclic graph $G_{\text{exp}} = (V, E)$ whose nodes are assignment literals $X_{r,c} = v$. The root is the target t, leaves correspond to givens in G, and internal nodes correspond to *admissible lemmas* introduced during construction. An edge (n, u) indicates that n belongs to the MUS used to justify u. Thus, incoming edges of a node record a local MUS-based justification for the corresponding literal.

Admissible lemmas. Starting from the base MUS G, we identify assignment literals that can already be derived from G and the Sudoku constraints C. A literal u is an *admissible lemma* for G if: (i) u belongs to the unique solution of the puzzle, and (ii) there exists a subset $M \subseteq G$ such that $C \cup M \cup \{\neg u\}$ is unsatisfiable. We denote by $\mathcal{L}(G)$ the set of all admissible lemmas for the base MUS G. Intuitively, admissible lemmas are intermediate consequences that can be justified directly from the same base MUS. This restriction prevents the construction from introducing arbitrary solution literals whose justification would require information outside the explanation context of the target. Let $\textsc{pool} = G \cup \mathcal{L}(G)$ denote the set of literals available during construction. All MUS computations in the recursive construction are performed with respect to $C \cup \textsc{pool}$.

Recursive construction. The explanation graph is constructed recursively. When expanding a literal u, we compute a MUS $M(u) \subseteq \textsc{pool}$ such that $C \cup M(u) \cup \{\neg u\}$ is unsatisfiable. For each $n \in M(u)$ an edge (n, u) is added. If $n \notin G$, the literal is recursively expanded. Algorithm 1 summarises the construction. The recursive subroutine Expand is given separately in Algorithm 2. The set Expanded prevents recursive expansion from following cycles. If expansion would introduce a cycle, the algorithm falls back to using only the base MUS G for the current literal.

Example. Figure 2 shows a lemma-based explanation graph generated automatically. The target literal $X_{1,4} = 5$ (green) is justified by two intermediate lemmas, $X_{3,4} = 4$ and $X_{6,6} = 5$ (yellow). The lemma $X_{3,4} = 4$ is derived directly from givens in the base MUS. The lemma $X_{6,6} = 5$ depends on another lemma, $X_{6,1} = 8$, which in turn depends on the givens $X_{5,8} = 8$, $X_{4,6} = 8$,

Algorithm 1 Recursive Lemma-Based Construction of an Explanation Graph

Input: hard constraints C, base MUS G, target literal t
Output: explanation graph $G_{\text{exp}} = (V, E)$
$V \leftarrow \emptyset$, $E \leftarrow \emptyset$
EXPANDED $\leftarrow \emptyset$
Compute admissible lemmas $\mathcal{L}(G)$
Call EXPAND(t)
return (V, E)

Algorithm 2 Recursive expansion subroutine EXPAND(u)

if $u \in$ EXPANDED **then**
  return
EXPANDED $\leftarrow$ EXPANDED $\cup \{u\}$
$V \leftarrow V \cup \{u\}$
Compute a MUS $M(u)$ of $C \cup$ POOL $\cup \{\neg u\}$
if $M(u) \cap$ EXPANDED $\neq \emptyset$ **then**
  Recompute $M(u)$ using only G
for each n in $M(u)$ **do**
  $V \leftarrow V \cup \{n\}$
  $E \leftarrow E \cup \{(n, u)\}$
  if $n \notin$ EXPANDED and $n \notin G$ **then**
    EXPAND(n)

and $X_{6,2} = 3$. The recursive expansion decomposes the flat MUS explanation of the target into nested justification steps, making explicit which intermediate deductions are reused in the explanation.

6 Rule-Based Explanation Graphs (Dual View)

Lemma-based explanation graphs describe how a target literal is derived from hints and intermediate lemmas. However, some MUSes consist almost entirely of givens and therefore do not decompose naturally into meaningful intermediate deduction steps.

To complement the lemma-based view, we introduce a *rule-based explanation view*. The two views organise the same contradiction differently. The lemma-based view focuses on reuse of literals and intermediate deductions, whereas the rule-based view focuses on reuse of structural Sudoku constraints.

Rule-based formulation. Let C denote the Sudoku constraints introduced in Section 3, H the hints, and t a target literal. Let $G \subseteq H$ be the base MUS obtained for t in the lemma-based explanation problem. In the rule-based view, the hints in G are treated as hard assumptions, while the structural Sudoku constraints are treated as removable rules. We then compute a smallest subset of constraints

$$U \subseteq C$$

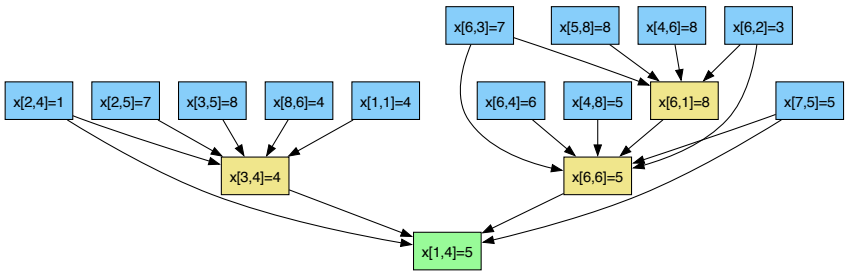

Fig. 2. Lemma-based explanation graph for the target literal $X_{1,4} = 5$. Blue nodes denote givens in the base MUS, yellow nodes denote introduced lemmas, and the green node denotes the target.

such that

$$G \cup \{\neg t\} \cup U$$

is unsatisfiable.

Intuitively, U identifies which structural Sudoku rules are jointly responsible for ruling out the counterfactual alternative $\neg t$.

Rule set. The candidate rules correspond exactly to the 27 `AllDifferent` constraints of the Sudoku model: the 9 row constraints, 9 column constraints, and 9 block constraints defined in Section 3. Cell-level exactly-one constraints remain part of the hard background theory and are not removed.

SMUS extraction. To obtain a minimum-cardinality rule explanation we again use the OptUx algorithm [11]. Each structural rule is represented by a selector literal controlling the CNF clauses corresponding to the associated `AllDifferent` constraint. Computing a smallest unsatisfiable subset of these selectors yields a minimum-cardinality set U of structural rules explaining the contradiction.

This grouping abstracts away from clause-level MUSes and instead produces explanations directly at the level of Sudoku rules such as rows, columns, and blocks.

Graph structure. The rule-based explanation is represented as a three-layer directed graph

$$\textit{Literals} \;\rightarrow\; \textit{Rules} \;\rightarrow\; \bot.$$

- **Layer 1:** hint literals together with $\neg t$,
- **Layer 2:** rule nodes corresponding to elements of U,
- **Layer 3:** a contradiction node $\bot$.

Edges exist only from literals to rules and from rules to $\bot$. A literal connects to a rule if the corresponding Boolean variable appears in the CNF encoding controlled by that rule selector. There are therefore no literal-to-literal or rule-to-rule edges.

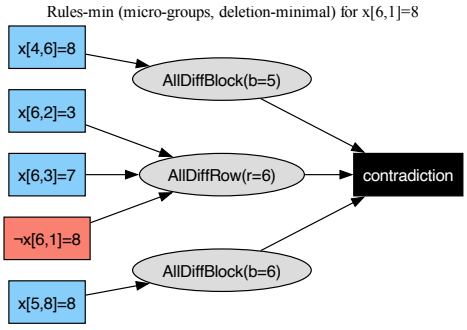

Fig. 3. Rule-based explanation graph.

Example. Figure 3 illustrates a rule-based explanation graph for the target literal $X_{6,1} = 8$. To justify this value, we consider the counterfactual assumption $X_{6,1} \neq 8$ (shown in red) together with the hints. The hints $X_{6,2} = 3$ and $X_{6,3} = 7$ together with $X_{6,1} \neq 8$ participate in the constraint $\texttt{AllDiffRow}(r = 6)$. Hint $X_{4,6} = 8$ together with $\texttt{AllDiffBlock}(b = 5)$ rules out value 8 for cells $(6, 4)$, $(6, 5)$, and $(6, 6)$. Similarly, hint $X_{5,8} = 8$ together with $\texttt{AllDiffBlock}(b = 6)$ rules out value 8 for cells $(6, 7)$, $(6, 8)$, and $(6, 9)$. Since row 6 must still contain an occurrence of value 8, the counterfactual assumption $X_{6,1} \neq 8$ becomes inconsistent. The rule-based explanation therefore highlights which structural Sudoku constraints jointly force the target literal.

7 Merged and Condensed Explanation Graphs

For each benchmark instance (assumed to admit a unique solution), we construct lemma-based explanation graphs for all non-hint solution literals. To analyse structural regularities across targets, these graphs are aggregated into a single dependency graph for the instance.

Merging. Let t denote a target literal and $G_t = (V_t, E_t)$ its lemma-based explanation graph. During merging, we retain only the incoming edges of the target node t and discard the deeper recursive structure. The resulting graph therefore records only the immediate justification dependencies associated with each target literal.

The merged graph is defined as

$$G_{\text{merge}} = \bigcup_t E_t^{\text{in}},$$

where E_t^{in} denotes the set of incoming edges of t. The resulting graph aggregates local justification dependencies across all targets of the instance.

Condensation. Although each individual explanation graph G_t is acyclic, their union may contain cycles. Such cycles arise when literals participate in mutually dependent justifications across different targets. To obtain a global partial-order representation, we compute the strongly connected components (SCCs) of G_{merge} and construct the corresponding condensation graph. Each SCC becomes a single node in the condensed graph, and edges between SCCs are induced by edges of G_{merge}. By construction, the condensation graph is a directed acyclic graph. The condensed graph therefore abstracts from cyclic local dependencies while preserving the global dependency structure between reasoning clusters.

Ranking. Each node of the condensation graph is assigned a rank equal to the length of its longest incoming path. This rank provides an approximate notion of dependency depth within the condensed dependency structure. It should not be interpreted as a unique solving order: multiple valid topological linearisations of the condensed graph may exist.

8 Bottom-Up Lemma-Based Explanation Construction

We present an alternative lemma-based construction that builds explanations bottom-up. Unlike the recursive, target-driven scheme of Algorithm 1, which expands explanations starting from a chosen target, this approach incrementally grows a set of established literals starting from the puzzle hints. We assume that the puzzle has a unique solution. Candidate lemmas therefore correspond to assignment literals associated with non-hint solution cells.

Iterative construction. Let G_0 denote the set of hints. At iteration k, let G_k denote the set of literals established before the iteration begins. For each non-hint cell (r, c) whose solution value is v^*, we compute a smallest MUS of $C \cup G_k \cup \{\neg x_{r,c,v^*}\}$. If the resulting SMUS satisfies the admissibility criterion described below, the literal x_{r,c,v^*} is marked for admission at rank $k+1$. All admissible candidates are collected during the iteration, but they are added to the established set only after the iteration has completed. Consequently, literals admitted at rank $k+1$ cannot support other literals admitted at the same rank. The process iterates until a fixpoint is reached. Since $G_{k+1} \supseteq G_k$ and the number of solution literals is finite, termination is guaranteed. For each admitted literal, the supporting literals appearing in its SMUS define the incoming edges of the explanation graph. The resulting structure is therefore layered according to the iteration at which literals become derivable from previously established facts.

Admissibility criterion. A literal is admitted only if its SMUS satisfies two conditions:

1. *bounded size:* $|\text{SMUS}| < 9$; The threshold reflects the intended granularity of local Sudoku reasoning. In particular, it admits explanations corresponding to the extreme case where a target placement is forced by the remaining eight cells of a row, column, or block. For example, if eight cells of a row already exclude all values except one, then the remaining cell becomes forced

by a support involving at most eight previously established literals together with the corresponding structural rule. We regard such explanations as still sufficiently local and cognitively manageable for human interpretation.
2. *rule-based connectivity:* in the associated rule-based explanation graph (Section 6), every rule node has an incoming edge from some literal in G_k.

Overall, the bottom-up scheme constructs layered dependency graphs in which each rank contains literals derivable from previously established layers. The resulting structure represents a partial dependency order: different topological linearisations may yield different valid explanation traces consistent with the same graph.

9 From Explanation Graphs to Explanation Traces

The explanation graphs introduced above provide a structured representation of MUS-based reasoning. Many explanation systems for constraint reasoning, however, present explanations as sequences of local deduction steps. For example, Demystify-style puzzle explanations are presented as traces of local MUS-based deductions [4]. To relate our framework to this style of explanation, we observe that our dependency graphs naturally induce explanation traces. The same trace extraction principle applies uniformly to both the Condensed and Bottom-up constructions introduced earlier. For merged graphs that contain cycles, traces are extracted after condensation into strongly connected components. In the remainder of this section, we focus on the trace extraction process for merged dependency graphs.

Trace extraction. Let G_{merge} be the merged dependency graph introduced in Section 7, and let G_{cond} be its condensation graph, whose nodes are the strongly connected components of G_{merge}. Although G_{merge} may contain cycles, G_{cond} is acyclic by construction. A literal-level explanation trace is obtained by processing the strongly connected components according to a topological ordering. For each component, we choose an arbitrary but fixed internal ordering of its literals, and concatenate these local orders. This yields a sequence

$$\tau = (u_1, \ldots, u_n)$$

of literals. If there is an edge from a component S_i to a component S_j in G_{cond}, then every literal in S_i appears before every literal in S_j in the resulting trace. Within a strongly connected component, literals belong to the same mutually dependent reasoning cluster, so no canonical internal order is imposed. The essential dependency information is therefore captured by the topological structure of the condensation graph, while the final trace is obtained through linearisation.

Each literal in the trace is associated with the MUS-based justification of that literal. The condensed graph therefore represents a global partial order over reasoning clusters, while a trace is a literal-level linearisation of that partial order.

Structured versus operational traces. A dependency graph exposes reusable intermediate deductions explicitly, whereas operational traces linearise reasoning into a sequence of local deduction steps. Different valid topological orders of the graph therefore induce different operational traces while preserving the same underlying dependency structure. For example, the graph in Figure 2 admits traces in which $X_{3,4} = 4$ and $X_{6,1} = 8$ are derived before $X_{6,6} = 5$, which in turn precedes the target $X_{1,4} = 5$. The graph representation compactly captures these dependencies while avoiding replication of shared support structure across multiple deduction steps.

A trace may also be interpreted as a sequence of state transitions. Starting from the initial puzzle state, each deduction step updates the current domains of the Sudoku variables, producing a progressively refined CSP state. This interpretation makes it possible to associate explanations not only with literals in the abstract dependency graph, but also with concrete transitions between accumulated trace states. In particular, explanations may be computed relative to the current trace state rather than relative only to the original puzzle hints.

Because each trace step updates the current variable domains, later deductions may admit smaller explanations than those recorded originally in the dependency graph. Trace-state explanations may therefore exploit accumulated domain information to reconstruct compact local justifications relative to the current CSP state.

The traces extracted from our dependency graphs currently linearise only established assignment literals. In contrast, operational traces produced by systems such as Demystify also include explicit candidate eliminations. These eliminations play an important role in the evolution of the trace state, since they progressively refine the variable domains from which later deductions become derivable.

One possible interpretation is that our dependency graphs provide a compact representation of a more verbose propagation graph in which candidate eliminations would also appear explicitly as nodes. Such an extended graph could contain edges of the form

$$(X_{i,j} = v) \rightarrow (X_{i,j} \neq v')$$

capturing propagation effects induced by established literals. A topological ordering of this richer graph would then produce a fully explicit operational trace containing both placements and candidate eliminations. However, simply materialising such elimination nodes would likely produce substantially larger traces and explanations. A more interesting direction for future work is therefore to extend the lemma-based and Bottom-up constructions themselves so that candidate eliminations may also be introduced as admissible intermediate deductions.

One qualitative motivation for the trace-based view is visible when comparing against Demystify-style operational traces. In the `cpcourse_1` trace, the placement $X_{3,1} = 7$ is explained as a *hidden single* relative to Column 1. This amounts to establishing that value 7 has been eliminated from the domains of all other cells in the column.

Our trace-state SMUS explanation reconstructs a smaller logical support for the same placement. Instead of using the full hidden-single support structure for the column, the SMUS identifies a subset of state facts ($X_{1,1} = 4$, $X_{6,3} = 7$, $X_{7,2} = 7$, and $X_{2,1} \in \{3, 5, 6, 9\}$) together with the structural constraints `AllDiffCol`(2), `AllDiffCol`(3), and `AllDiffBlock`(1), which already suffice to force the placement. In this sense, the explanation still captures a hidden-single style reasoning pattern, but through a smaller MUS-derived support set than the one produced in the corresponding Demystify explanation.

10 Empirical Evaluation

This section evaluates both the computational behaviour and the structural properties of the proposed explanation constructions on reconstructed Demystify traces. We use the same native CP backend throughout the main experiments: a C++/Gecode oracle with AFC-size search, together with the portfolio SMUS procedure described in the accompanying native-CP implementation notes. We study: (i) QuickXplain versus the native Gecode portfolio on the same placement targets, (ii) structural differences between Condensed and Bottom-up dependency graphs, and (iii) the relationship between graph-based explanations and Demystify-style operational traces.

10.1 Experimental Setup

Experiments were conducted on a MacBook Pro (M1 Pro, 32 GB RAM) running macOS, using Python 3.11. The CP oracle was implemented natively in C++ using Gecode 6.2.0. Native oracle calls use AFC-size search, including in the QuickXplain baseline. Minimum-cardinality supports were computed by a native Gecode portfolio over hitting-set search configurations. Earlier versions of this work used OptUx for SMUS extraction; in the experiments reported here we use the native portfolio because separate performance experiments, not included in this report, showed it to be substantially faster on the relevant Sudoku workloads. The portfolio implementation follows the same high-level ideas as OptUx, combining bootstrapping, shrinking, and hitting-set exploration. It can therefore be viewed as materialising the CP-oracle implementation route anticipated by Ignatiev et al. [11], while instantiating the consistency oracle with our native CP/Gecode backend. Unless stated otherwise, runtimes are wall-clock times.

The benchmark set contains thirteen reconstructed Demystify-style traces: the `cpcourse_1` trace and the 12 BremSter hard-instance traces. For the trace-state experiments, each non-given deduction was given a 10-second per-target timeout; givens are reported as unit supports. The explanation-item vocabulary for trace-state SMUSes consists of trace-prefix facts, compact domain hints, and Sudoku `AllDiff` row, column, and block rule instances.

10.2 MUS Extraction: QuickXplain vs Native Gecode Portfolio

We compare two MUS-extraction strategies on the same set of placement targets from the thirteen benchmark instances:

- **QuickXplain (QX):** computes subset-minimal MUSes.
- **Native Gecode portfolio:** computes minimum-cardinality MUSes using the native CP oracle and portfolio hitting-set search.

Both approaches use the same native AFC-size Gecode oracle for consistency checks, isolating the difference between subset-minimal and minimum-cardinality extraction. Figure 4 plots runtime against the support size produced by each method. Across the 724 shared placement targets, QuickXplain has median runtime 1.21 seconds, whereas the native Gecode portfolio has median runtime 0.026 seconds on the same target set. The portfolio computes cardinality-minimal supports, while QuickXplain computes subset-minimal supports. Thus, when the portfolio is run with exact workers and without a timeout, its support for a given target is no larger than the corresponding QuickXplain support, although the two supports need not contain the same items. Runtime is nevertheless not determined by support size alone: large variation remains, including a few portfolio outliers, because the cost of both procedures also depends on the structure of the oracle calls and, for the portfolio, on the hitting-set search.

In the subsequent structural experiments we use the native Gecode portfolio throughout, providing a consistent minimum-cardinality basis for the Condensed, Bottom-up, and trace-state analyses.

10.3 Condensed vs Bottom-Up

We compare the two explanation constructions introduced in Sections 7 and 8 on the same thirteen benchmark instances. For each instance, we analyse the distribution of node in-degree, corresponding to support size, across ranks in the dependency graph (Figure 5).

Structural behaviour. The Condensed construction is target-driven: each target is explained independently and the resulting target dependencies are merged and condensed. This can produce broad variation in support size across ranks. The Bottom-up construction instead admits literals iteratively, only when their explanations satisfy the bounded-size and rule-connectivity criteria. It therefore tends to produce a more layered structure in which deductions at later ranks are justified from previously admitted literals. We also report an ablation, *Bottom-up no connectivity*, in which the same bounded-size literal supports are used but the rule-core connectivity check is omitted. This isolates how much of the graph shape and runtime is due to the connectivity condition itself.

Computational cost. Figure 6 compares end-to-end runtime for the three graph-construction variants, including the common base-support computation performed before graph construction starts. Condensed construction computes target-driven lemma graphs, whereas the connected Bottom-up construction repeatedly tests candidate literals for admissibility until reaching a fixpoint. With

the native CP portfolio backend, connected Bottom-up is faster than Condensed on 11 of the 13 benchmark instances. This reflects the effect of the admissibility criteria: they can quickly rule out difficult targets and avoid constructing large target-driven lemma graphs. The two exceptions, where Condensed is faster, show that the relative cost is still instance-dependent; Bottom-up may spend more time when many candidate explanations have to be reconsidered across ranks.

The no-connectivity ablation is faster than both connected Bottom-up and Condensed on all 13 instances, confirming that the rule-core connectivity layer is a significant source of runtime. The coarse graph summaries are nevertheless very similar to connected Bottom-up: the number of admitted and non-admitted targets is unchanged on all instances, and the maximum rank changes only for BremSter95. Thus, in this benchmark set the connectivity check is computationally relevant while having limited impact on these coarse graph statistics. We keep the connected Bottom-up construction as the main method because it retains the intended rule-based structural guarantee.

10.4 Comparison with Demystify-Style Trace Explanations

To better understand the relationship between our framework and Demystify-style trace explanations, we analysed explanations relative to reconstructed trace states derived from Demystify sessions.

Demystify directly produces explicit linear traces containing both placement deductions and candidate eliminations. In contrast, our framework constructs dependency graphs that represent partial orders between deductions. Concrete traces can then be obtained by selecting topological linearisations of these dependency graphs.

We compare the partial orders induced by the explanation graphs with the total orders appearing in the corresponding Demystify traces. For each precedence relation induced by the transitive closure of an explanation graph, we check whether the source deduction appears no later than the target deduction in the linear Demystify trace. This transitive-closure check subsumes the check on direct graph edges. Across the thirteen benchmark instances, the lemma-based graphs respect 95.9% of these transitive-closure precedence relations. The Bottom-up graphs show an even stronger agreement on the smaller set of admitted bounded explanations, respecting 98.6% of the corresponding precedence relations. This suggests that the dependency structure captured by the graph-based constructions is highly compatible with the deduction structure underlying the corresponding Demystify traces.

Table 1 summarises the size of the reconstructed Demystify traces for the benchmark instances considered in this section. For each instance, we report the total number of deductions together with the breakdown between placement deductions and candidate eliminations. The reconstructed traces are dominated by candidate eliminations rather than placement deductions, illustrating the operational verbosity of fully linearised traces.

Table 1. Summary of reconstructed Demystify trace lengths.

Instance	Total deductions	Placements	Eliminations
cpcourse_1	537	81	456
BremSter95	537	81	456
BremSter103	553	81	472
BremSter104	569	81	488
BremSter109	521	81	440
BremSter116	513	81	432
BremSter121	513	81	432
BremSter126	537	81	456
BremSter131	489	81	408
BremSter136	521	81	440
BremSter137	545	81	464
BremSter141	537	81	456
BremSter142	473	81	392

We additionally analysed the support sizes and runtimes of native trace-state SMUS explanations for placement and elimination deductions. For each trace step, explanations were computed relative to the current reconstructed CSP state induced by the preceding deductions in the trace. Figures 7 and 8 compare the corresponding support-size and runtime distributions across the thirteen benchmark instances. Deductions for which the native portfolio did not return an exact support within the timeout are omitted from both boxplots. Candidate eliminations generally admit smaller and less variable supports, reflecting the fact that they often arise from highly local conflicts. Placement deductions typically require combining several previously established state facts together with multiple structural constraints in order to show that all alternative values have been excluded. Runtime is reported in seconds; placements and eliminations have similar median runtimes in these experiments, but placement explanations exhibit higher variance.

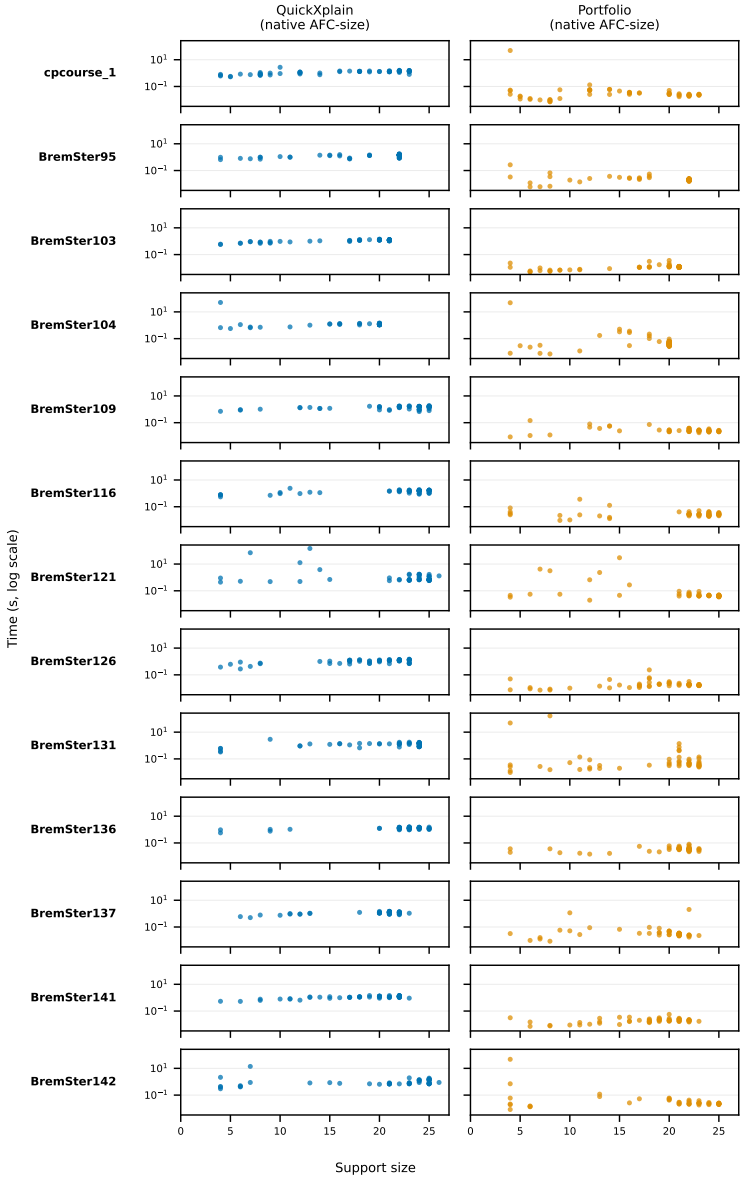

Fig. 4. Runtime versus method-specific support size for QuickXplain with the native AFC-size Gecode oracle and for the native Gecode portfolio. Rows correspond to benchmark instances and columns to extraction methods. All panels use a log-scale runtime axis.

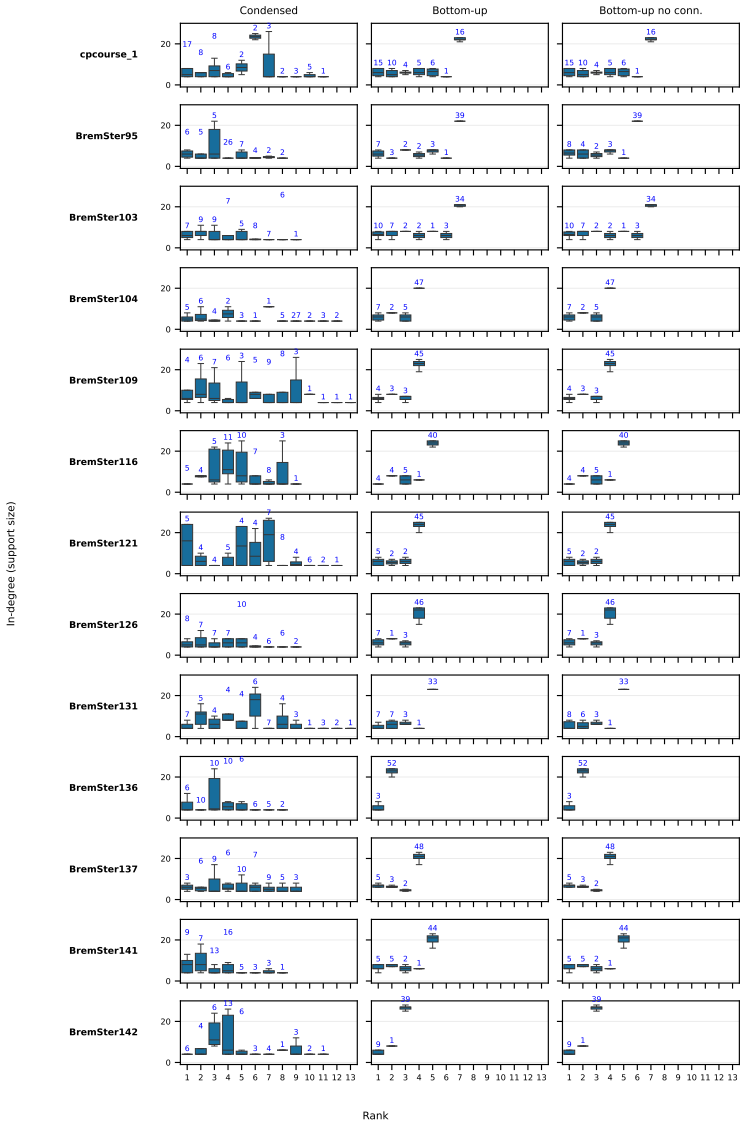

Fig. 5. Distribution of node in-degree (support size) by rank for the Condensed, Bottom-up, and Bottom-up no-connectivity constructions across the thirteen benchmark instances. Rows correspond to instances; columns correspond to methods. Blue annotations give the number of nodes represented by each box. In the connected Bottom-up panels, targets not admitted by the bounded-size and rule-connectivity criteria are plotted at their terminal rank using the cardinality of their base support; in the no-connectivity ablation, the analogous terminal-rank targets are those not admitted by the bounded-size literal-support test alone.

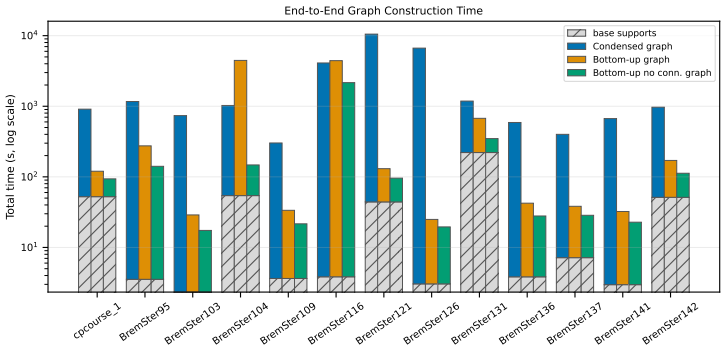

Fig. 6. End-to-end runtime comparison, in seconds, between Condensed, connected Bottom-up, and Bottom-up no-connectivity constructions on the thirteen benchmark instances (log scale). Stacked bars separate the common base-support computation from the approach-specific graph-construction phase.

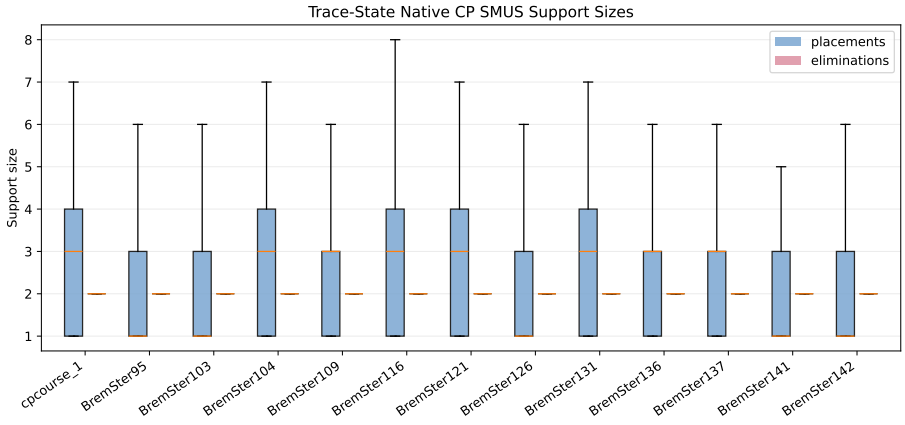

Fig. 7. Distribution of trace-state SMUS support sizes for placement and elimination deductions across the thirteen benchmark instances. Elimination explanations tend to be smaller and exhibit lower variance than placement explanations.

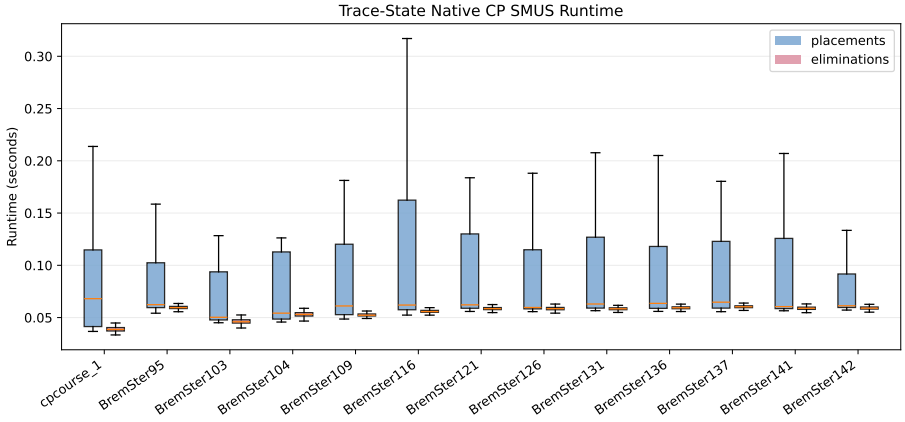

Fig. 8. Runtime, in seconds, for exact native trace-state SMUS explanations of placement and elimination deductions across the thirteen benchmark instances. Placement explanations have similar median runtime to elimination explanations, but show higher variance.

11 Conclusion

We presented a structured framework for deriving explanation graphs from minimal unsatisfiable subsets (MUSes) in Sudoku. Rather than treating MUSes as flat explanation objects, our approach organises them into dependency structures that expose intermediate deductions, support reuse, and structural relations between reasoning steps.

The framework combines two complementary perspectives. In the *lemma-based view*, hints are treated as soft constraints and explanations correspond to MUSes of hints with respect to the Sudoku rules. This view supports hierarchical explanation graphs constructed either through a target-driven Condensed procedure or through an iterative Bottom-up admission scheme.

In the complementary *rule-based view*, the roles of rules and hints are exchanged, allowing explanations to be expressed directly in terms of the structural Sudoku constraints responsible for a contradiction. This rule-oriented perspective provides a structural abstraction layer above clause-level MUSes and highlights the interaction between rows, columns, and blocks in the derivation of a target.

We further showed that these explanation graphs naturally induce explanation traces. The resulting dependency graphs represent partial orders between deductions, while concrete traces are obtained through topological linearisations of these structures. This distinction makes it possible to relate graph-based explanations to operational trace systems such as Demystify.

Our empirical analysis showed strong compatibility between the partial orders induced by our dependency graphs and the operational deduction orders generated by Demystify traces. The trace-state evaluation also showed that placement explanations are more complex than elimination explanations: their native high-level CP supports are typically larger and exhibit higher variance. The graph representation complements fully linearised traces by making shared dependency structure explicit rather than repeating it through many local propagation steps.

The comparison between Condensed and Bottom-up constructions revealed a trade-off between computational cost and structural regularity. The Condensed construction gives a target-driven dependency view, whereas the connected Bottom-up construction enforces bounded-size and rule-connectivity admissibility criteria that produce more uniform and structurally coherent dependency layers. On the thirteen reconstructed Demystify benchmark instances, connected Bottom-up is faster than Condensed in 11 cases under the native CP portfolio backend, because admissibility filtering often avoids difficult target-driven graph construction. A no-connectivity ablation is faster than connected Bottom-up on all instances and preserves the same admitted/non-admitted counts in this benchmark set, but it deliberately drops the rule-based connectivity guarantee. The remaining cases show that the relative runtime of the principled connected construction is still instance-dependent.

SMUS extraction in the main empirical evaluation is performed with a native CP backend: consistency checks are delegated to Gecode and the minimum-cardinality search is handled by the portfolio hitting-set procedure. This keeps

the experimental story within the CP model while still exposing the MUS/SMUS structure needed by the explanation graph constructions.

In the intended application scenario, explanation graphs are computed offline for a given puzzle instance and subsequently explored interactively. The reported runtimes therefore correspond primarily to preprocessing cost rather than latency experienced during interactive explanation exploration.

Overall, the results demonstrate that explanation quality is not determined solely by support minimality, but also by how supports are organised, linearised, and reused across deductions. The proposed framework therefore provides a bridge between MUS-based explanation systems, dependency-graph representations, and operational trace explanations.

Future work includes extending the framework beyond Sudoku, integrating candidate eliminations directly into the graph construction process, studying finer-grained rule decompositions, and exploring the relationship between MUS-based explanations and propagation-based local consistency reasoning.

References

1. Bogaerts, B., Gamba, E., Guns, T.: A framework for step-wise explaining how to solve constraint satisfaction problems. Artif. Intell. **300**, 103550 (2021). https://doi.org/10.1016/J.ARTINT.2021.103550, https://doi.org/10.1016/j.artint.2021.103550
2. Cambazard, H., Jussien, N.: Identifying and exploiting problem structures using explanation-based constraint programming. Constraints An Int. J. **11**(4), 295–313 (2006). https://doi.org/10.1007/S10601-006-9002-8, https://doi.org/10.1007/s10601-006-9002-8
3. Eppstein, D.: Solving single-digit sudoku subproblems. In: International Conference on Fun with Algorithms. pp. 142–153. Springer (2012)
4. Espasa, J., Gent, I.P., Hoffmann, R., Jefferson, C., Lynch, A.M., Salamon, A., McIlree, M.J.: Using small muses to explain how to solve pen and paper puzzles. arXiv preprint arXiv:2104.15040 (2021)
5. Feydy, T., Stuckey, P.J.: Lazy clause generation reengineered. In: Gent, I.P. (ed.) Principles and Practice of Constraint Programming - CP 2009, 15th International Conference, CP 2009, Lisbon, Portugal, September 20-24, 2009, Proceedings. Lecture Notes in Computer Science, vol. 5732, pp. 352–366. Springer (2009). https://doi.org/10.1007/978-3-642-04244-7_29, https://doi.org/10.1007/978-3-642-04244-7_29
6. Gamba, E., Bogaerts, B., Guns, T.: Efficiently explaining csps with unsatisfiable subset optimization. In: Zhou, Z. (ed.) Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021. pp. 1381–1388. ijcai.org (2021). https://doi.org/10.24963/IJCAI.2021/191, https://doi.org/10.24963/ijcai.2021/191
7. Gocht, S., McCreesh, C., Nordström, J.: An auditable constraint programming solver. In: Solnon, C. (ed.) 28th International Conference on Principles and Practice of Constraint Programming, CP 2022, Haifa, Israel, July 31 - August 8, 2022. LIPIcs, vol. 235, pp. 25:1–25:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022). https://doi.org/10.4230/LIPICS.CP.2022.25, https://doi.org/10.4230/LIPIcs.CP.2022.25

8. Guns, T., Gamba, E., Mulamba, M., Bleukx, I., Berden, S., Pesa, M.: Sudoku assistant - an ai-powered app to help solve pen-and-paper sudokus. In: Williams, B., Chen, Y., Neville, J. (eds.) Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023. pp. 16440–16442. AAAI Press (2023). https://doi.org/10.1609/AAAI.V37I13.27072, https://doi.org/10.1609/aaai.v37i13.27072
9. Gupta, S.D., Genc, B., O'Sullivan, B.: Explanation in constraint satisfaction: A survey. In: Zhou, Z. (ed.) Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021. pp. 4400–4407. ijcai.org (2021). https://doi.org/10.24963/IJCAI.2021/601, https://doi.org/10.24963/ijcai.2021/601
10. Hoffmann, R., Zhu, X., Akgün, Ö., Nacenta, M.A.: Understanding how people approach constraint modelling and solving. In: Solnon, C. (ed.) 28th International Conference on Principles and Practice of Constraint Programming, CP 2022, Haifa, Israel, July 31 - August 8, 2022. LIPIcs, vol. 235, pp. 28:1–28:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022). https://doi.org/10.4230/LIPICS.CP.2022.28, https://doi.org/10.4230/LIPIcs.CP.2022.28
11. Ignatiev, A., Previti, A., Liffiton, M.H., Marques-Silva, J.: Smallest MUS extraction with minimal hitting set dualization. In: Pesant, G. (ed.) Principles and Practice of Constraint Programming - 21st International Conference, CP 2015, Cork, Ireland, August 31 - September 4, 2015, Proceedings. Lecture Notes in Computer Science, vol. 9255, pp. 173–182. Springer (2015). https://doi.org/10.1007/978-3-319-23219-5_13, https://doi.org/10.1007/978-3-319-23219-5_13
12. Junker, U.: QUICKXPLAIN: preferred explanations and relaxations for over-constrained problems. In: McGuinness, D.L., Ferguson, G. (eds.) Proceedings of the Nineteenth National Conference on Artificial Intelligence, Sixteenth Conference on Innovative Applications of Artificial Intelligence, July 25-29, 2004, San Jose, California, USA. pp. 167–172. AAAI Press / The MIT Press (2004), http://www.aaai.org/Library/AAAI/2004/aaai04-027.php
13. Korekawa, T., Komiya, K., Kotani, Y.: " solution path": The structure of the solution steps for pencil puzzle. In: 2010 International Conference on Technologies and Applications of Artificial Intelligence. pp. 217–221. IEEE (2010)
14. Liffiton, M.H., Previti, A., Malik, A., Marques-Silva, J.: Fast, flexible MUS enumeration. Constraints An Int. J. **21**(2), 223–250 (2016). https://doi.org/10.1007/S10601-015-9183-0, https://doi.org/10.1007/s10601-015-9183-0
15. Liffiton, M.H., Sakallah, K.A.: Algorithms for computing minimal unsatisfiable subsets of constraints. J. Autom. Reason. **40**(1), 1–33 (2008). https://doi.org/10.1007/S10817-007-9084-Z, https://doi.org/10.1007/s10817-007-9084-z
16. Lynce, I., Ouaknine, J.: Sudoku as a SAT problem. In: International Symposium on Artificial Intelligence and Mathematics, AI&Math 2006, Fort Lauderdale, Florida, USA, January 4-6, 2006 (2006), http://anytime.cs.umass.edu/aimath06/proceedings/P34.pdf
17. Ohrimenko, O., Stuckey, P.J., Codish, M.: Propagation via lazy clause generation. Constraints An Int. J. **14**(3), 357–391 (2009). https://doi.org/10.1007/S10601-008-9064-X, https://doi.org/10.1007/s10601-008-9064-x
18. Reiter, R.: A theory of diagnosis from first principles. Artif. Intell. **32**(1), 57–95 (1987). https://doi.org/10.1016/0004-3702(87)90062-2, https://doi.org/10.1016/0004-3702(87)90062-2

19. Rossi, F., van Beek, P., Walsh, T. (eds.): Handbook of Constraint Programming, Foundations of Artificial Intelligence, vol. 2. Elsevier (2006), https://www.sciencedirect.com/science/bookseries/15746526/2
20. Simonis, H.: Sudoku as a constraint problem. In: Modelling and Reformulating Constraint Satisfaction Problems (CP Workshops 2005) (2005)

A Explanation of Positive Targets

Figure 1 in the introduction illustrates our visual representation for explanations. In this appendix we provide additional examples together with a more detailed description of the visual encoding. Each figure represents an explanation for a target cell in the Sudoku grid. Cell identifiers are shown in the bottom-right corner of each cell. We use two equivalent conventions to refer to cells: the alphanumeric notation used in Sudoku (e.g. 1E) and the corresponding numeric cell identifier used in the figures (e.g. 5). These refer to the same grid position. The target cell is outlined in orange and the value being proved is indicated by a circled digit. Hints participating in the explanation (either givens or previously derived lemmas of lower rank) are also outlined in orange. Constraints containing the target cell are rendered with a transparent green background. Constraints not containing the target cell are rendered with a transparent orange background. Such constraints must interact with at least one other participating constraint. Every hint used in the explanation appears in at least one of these constraints. For lemma cells, the deduced value is shown in orange inside the cell. The label in the lower-left corner indicates the lemma rank together with the number of hints used in its explanation. The example shown in Figure 1 corresponds to target cell 5. Additional examples are shown in Figures 9–11.

We begin with explanations establishing positive target values. Figure 9 illustrates an explanation establishing $X_{4,A} = 1$. Rule `AllDiffRow`(4) together with hints $X_{4,C} = 6$, $X_{4,F} = 8$, and $X_{4,H} = 5$ excludes values 6, 8, and 5 from $X_{4,A}$. Rule `AllDiffBlock`(4) together with hints $X_{5,B} = 9$, $X_{6,B} = 3$, and $X_{7,C} = 7$ excludes values 9, 3, and 7 from $X_{4,A}$. Rule `AllDiffCol`(A) together with hints $X_{1,A} = 4$ and $X_{7,A} = 2$ excludes values 4 and 2 from $X_{4,A}$. These exclusions leave 1 as the only remaining value for $X_{4,A}$. Therefore $X_{4,A} = 1$.

Figure 10 shows an explanation establishing $X_{6,F} = 5$. From `AllDiffBlock`(6) together with hints $X_{4,H} = 5$ and $X_{5,H} = 8$, the values 5 and 8 must appear in row 6 outside Block 6. Since row 6 already contains the hints $X_{6,B} = 3$, $X_{6,C} = 7$, and $X_{6,D} = 6$, the only relevant cells are $X_{6,A}$, $X_{6,E}$, and $X_{6,F}$. Rule `AllDiffCol`(E) together with hint $X_{7,E} = 5$ excludes value 5 from $X_{6,E}$. Rule `AllDiffBlock`(5) together with hint $X_{4,F} = 8$ excludes value 8 from $X_{6,E}$. Hence $X_{6,E} = 2$. Finally, `AllDiffBlock`(5) together with $X_{4,F} = 8$ excludes value 8 from $X_{6,F}$. Since the values 5 and 8 must appear among $X_{6,A}$, $X_{6,E}$, and $X_{6,F}$, and since $X_{6,E} = 2$, it follows that $X_{6,F} = 5$.

Figure 11 shows an explanation establishing $X_{8,D} = 2$. Row 8 intersects Block 7 in cells $X_{8,A}$, $X_{8,B}$, and $X_{8,C}$, and intersects Block 9 in cells $X_{8,G}$, $X_{8,H}$, and $X_{8,I}$. Rule `AllDiffBlock`(7) together with hint $X_{7,A} = 2$ excludes value 2

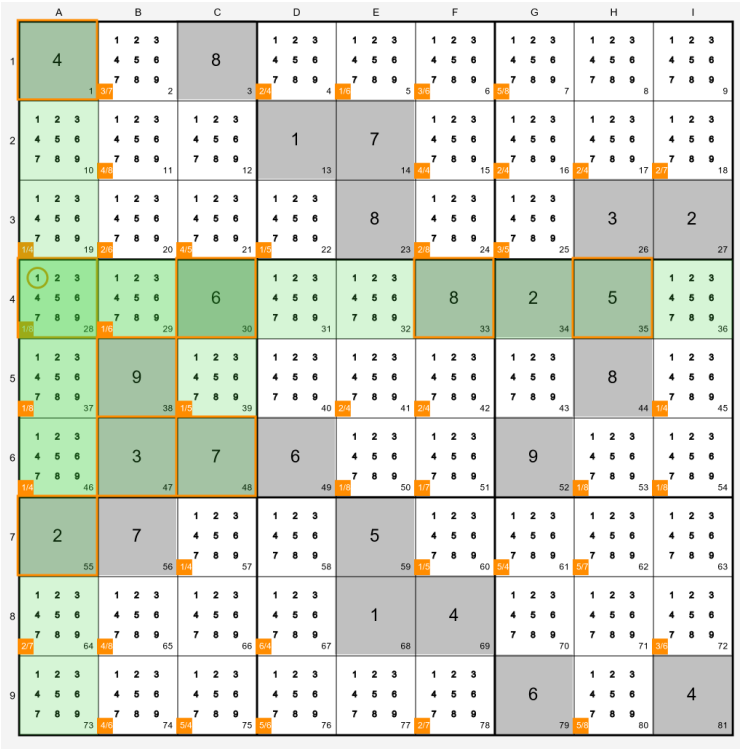

Fig. 9. Explanation establishing $X_{4,A} = 1$.

from $X_{8,A}$, $X_{8,B}$, and $X_{8,C}$. Similarly, rule `AllDiffBlock`(9) together with hint $X_{9,H} = 2$ excludes value 2 from $X_{8,G}$, $X_{8,H}$, and $X_{8,I}$. Hence, in row 8, value 2 can only appear in one of $X_{8,D}$, $X_{8,E}$, or $X_{8,F}$. Rule `AllDiffRow`(8) together with hints $X_{8,E} = 1$ and $X_{8,F} = 4$ then leaves $X_{8,D}$ as the only remaining position for value 2. Therefore $X_{8,D} = 2$.

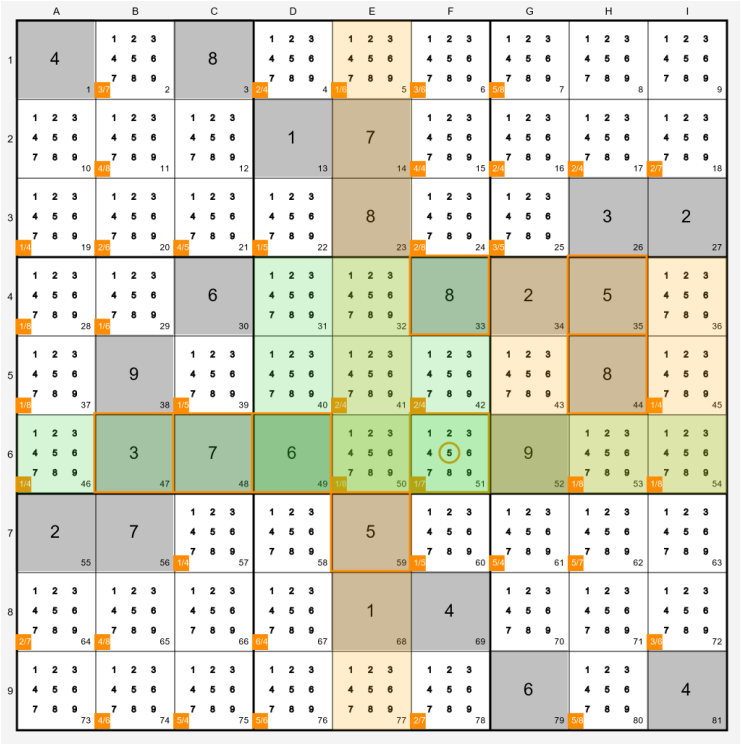

Fig. 10. Explanation establishing $X_{6,F} = 5$.

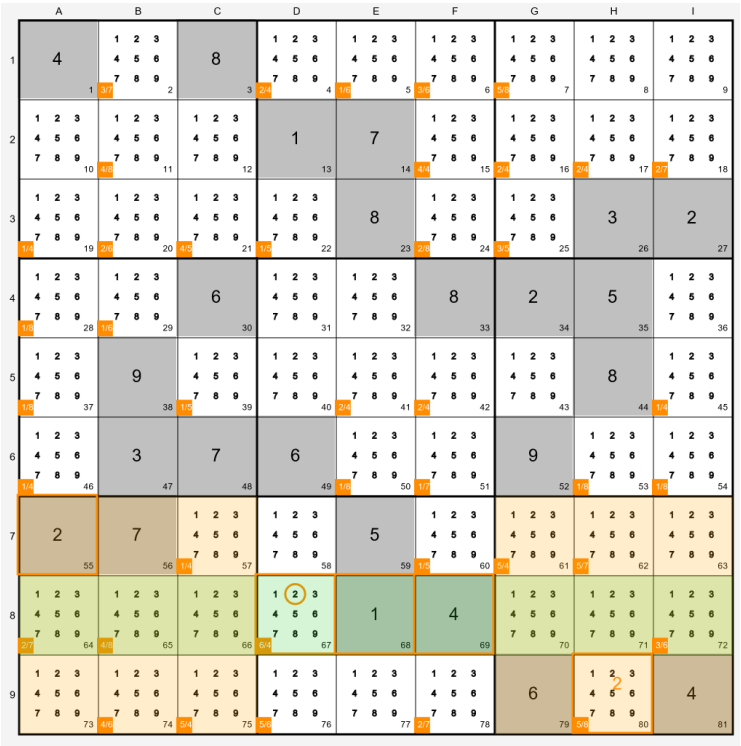

Fig. 11. Explanation establishing $X_{8,D} = 2$.

B Explanation of Negative Targets

Figure 12 illustrates an explanation refuting the assignment $X_{4,I} = 3$. Unlike the positive-target explanations in the previous section, this explanation relies on accumulated domain information represented through derived domain restrictions.

The participating domain restrictions are

$$X_{4,E} \in \{3, 9\}, \quad X_{7,D} \in \{3, 9\}, \quad X_{7,I} \in \{3, 9\}, \quad X_{9,E} \in \{3, 9\}.$$

The structural constraints participating in the explanation are

$$\texttt{AllDiffRow}(4), \quad \texttt{AllDiffRow}(7),$$
$$\texttt{AllDiffCol}(E), \quad \texttt{AllDiffCol}(I), \quad \texttt{AllDiffBlock}(8).$$

Assume $X_{4,I} = 3$. By $\texttt{AllDiffRow}(4)$, value 3 cannot appear elsewhere in row 4, and therefore $X_{4,E} \neq 3$. By $\texttt{AllDiffCol}(I)$, value 3 cannot appear elsewhere in column I. Since the domain restriction $X_{7,I} \in \{3, 9\}$ allows only values 3 and 9, it follows that $X_{7,I} = 9$. Now consider $\texttt{AllDiffRow}(7)$. Since $X_{7,I} = 9$, the restriction $X_{7,D} \in \{3, 9\}$ forces $X_{7,D} = 3$. Next, $\texttt{AllDiffCol}(E)$ excludes value 3 from the remaining cells of column E. Because $X_{7,D} = 3$ lies in Block 8, constraint $\texttt{AllDiffBlock}(8)$ excludes value 3 from the other cells of that block. Consequently, the restriction $X_{9,E} \in \{3, 9\}$ forces $X_{9,E} = 9$. Finally, $\texttt{AllDiffCol}(E)$ excludes value 9 from $X_{4,E}$. Since the restriction $X_{4,E} \in \{3, 9\}$ limits the domain of this cell to $\{3, 9\}$, no admissible value remains for $X_{4,E}$, yielding a contradiction. Therefore the assumption $X_{4,I} = 3$ is inconsistent, and we conclude $X_{4,I} \neq 3$.

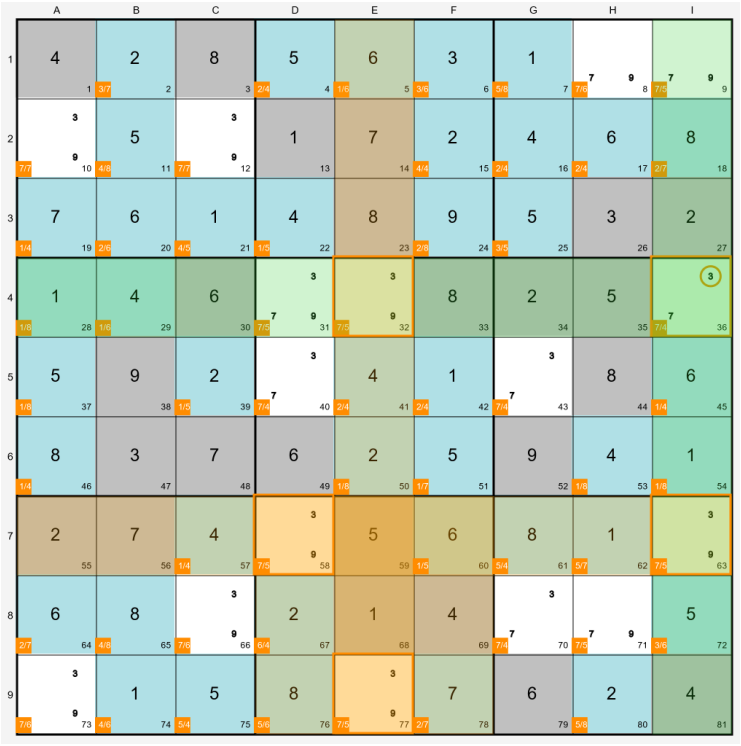

Fig. 12. Explanation refuting the assignment $X_{4,I} = 3$.

C Selected WPF/BremSter Puzzle Definitions

The file `hard_instances/selected.json` records the World Puzzle Federation Sudoku Grand Prix puzzles used as the BremSter component of the benchmark set. It contains one JSON object per puzzle. The field `source` identifies the presentation source; in this file all entries are labelled `BremSter`. The field `nr` is the BremSter puzzle identifier used in the experiments. The fields `grade`, `points`, `title`, and `author` describe the original WPF Grand Prix source and score. The fields `link` and `duration` record the corresponding video reference. The field `comment` is a selection note used during curation and is not used by the explanation algorithms. The field `rules` contains the puzzle rules; all selected puzzles are standard Sudoku instances. Finally, `grid` is a 9×9 array of strings: digits denote fixed givens and `_` denotes an empty cell.

Table 2 summarises the selected puzzles and the metadata recorded with them. The actual grid definitions are stored in the JSON file as row-major 9×9 arrays, using digits for givens and `_` for empty cells. The thirteenth benchmark instance, `cpcourse_1`, is the course example used throughout the paper and is stored separately from this BremSter/WPF selection file. In the table, *WPF source* gives the Grand Prix year and round, *Title* is the title used in the Brem-

Table 2. Selected WPF/BremSter standard Sudoku instances.

Instance	WPF source	Title	Points	Givens	Video
BremSter95	2017 R3	Classic Sudoku 5	54	24	26:11
BremSter103	2017 R2	Classic Sudoku 4	20	22	14:01
BremSter104	2017 R2	Classic Sudoku 5	41	20	16:01
BremSter109	2017 R1	Classic Sudoku 5	49	26	17:01
BremSter116	2015 R3	Classic Sudoku 6	55	27	24:07
BremSter121	2015 R4	Classic Sudoku 5	65	27	26:42
BremSter126	2015 R5	Classic Sudoku 5	55	24	17:26
BremSter131	2015 R6	Classic Sudoku 5	45	30	20:00
BremSter136	2015 R7	Classic Sudoku 5	55	26	25:26
BremSter137	2015 R7	Classic Sudoku 6	70	23	33:41
BremSter141	2015 R8	Classic Sudoku 4	45	24	13:47
BremSter142	2015 R8	Classic Sudoku 5	45	32	19:25

Ster video, *Points* is the WPF score, *Givens* is the number of fixed cells in the initial grid, and *Video* is the duration of the referenced BremSter solve video.